Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Devices, Interrupts, and System Calls **5**

1.1 Unit 1: Devices and a Polled Driver 7
1.1.1 CMSDK and CMSIS: Hardware versus Software 7
1.1.2 The Emulated Board: MPS2 with AN385 7
1.1.3 Hardware Architecture: AHB vs. APB Buses 8
1.1.4 The Board's Interrupt Lines 8
1.1.5 Software Organization: `include/cmsdk.h` 8
1.1.6 The Board's Memory Map from QEMU 9
Interpreting the Output: Real Models vs. Placeholders 9
Touching a Device That Is Not There 10
1.1.7 The General I/O Landscape 10
1.1.8 DMA: Moving Data Without the CPU 11
1.1.9 Memory-Mapped I/O: Loads and Stores with Side Effects . . . 11
1.1.10 The `volatile` Keyword 12
Try It: `volatile` at `-O2` 12
1.2 Read-Modify-Write Needs Care 13
1.2.1 Safer Hardware Patterns 14
1.3 The Device Protocol and the Driver Concept 14
1.3.1 The CMSDK UART Registers 15
Configuring the Baud Rate 15
1.4 Implementing the Device Driver 16
1.4.1 How Bigger OSes Generalize a Driver 16
1.4.2 Implementing and Using the UART Driver 16
Testing the Driver 17
1.4.3 Try It: A Polled Echo 17
1.5 Variadic Functions and `<stdarg.h>` 18
1.5.1 Separating Formatting from Output (`vformat`) 19
1.5.2 Writing `vformat` 19
Why `0u - (uint32_t) v` and not `-v`? 21
1.6 The Kernel Print Function (`kprintf`) 21
1.6.1 Kernel Mode vs. User Mode Printing 21
1.6.2 Implementing `kprintf` 21
1.7 Upgrading the Kernel: Fault Handlers That Print 22
1.7.1 The Status Registers (HFSR and CFSR) 22
1.7.2 Implementing the New Handlers 23
1.7.3 Try It: Catching a Divide-By-Zero 23
1.7.4 Practical Note: Project Navigation and Exam Expectations . 24
1.8 Unit 2: Calling Convention and Interrupts 24

1.8.1 Polling versus Interrupts . . . 24
1.8.2 The AAPCS: How C Calls a Function . . . 25
1.8.3 Writing Assembly Callable from C: The CPU's Interrupt State 25
The Nesting Problem . . . 25
Implementing `irq_save` and `irq_restore` in Assembly . . . 26
Code Breakdown: . . . 26
Testing the API in C . . . 27
Watching the AAPCS Across the Call (GDB) . . . 27
1.9 ARM Exception Entry and Return Mechanics . . . 27
1.9.1 The Automatic Hardware State Save (Stacking) . . . 28
1.9.2 Exception Return: `EXC_RETURN` . . . 28
1.9.3 Tail-Chaining and Late Arrival . . . 29
1.10 The Heartbeat of the OS: The SysTick Timer . . . 29
1.10.1 Hardware Preemption . . . 29
1.10.2 The SysTick Hardware Registers . . . 30
1.10.3 Building the SysTick Driver . . . 30
Choosing the Tick: Reload Value and Rate . . . 30
Uptime in Milliseconds and Wraparound Safety . . . 31
`wfi`: An Idle Loop That Sleeps . . . 32
1.11 The Nested Vectored Interrupt Controller (NVIC) . . . 32
1.11.1 The Three Gates and the Golden Rule . . . 33
1.12 Expanding the Vector Table for External Devices . . . 33
1.13 Concurrency and Shared Data . . . 34
1.13.1 Data Shared by a Handler and `main` . . . 34
1.13.2 Critical Sections with `irq_save` and `irq_restore` . . . 34
1.13.3 A Ring Buffer . . . 34
1.13.4 Interrupt-Driven Transmission . . . 35
The Producer: `uart_write` . . . 35
The Consumer: `UART0TX_Handler` . . . 36
Initialization and Priority . . . 36
Testing the Ring Buffer: Sending 160 Bytes . . . 37
Execution Analysis . . . 37
Verification via GDB . . . 37
1.14 Unit 3: System Calls and the Kernel Boundary . . . 38
1.14.1 Recap: Privilege and `svc` . . . 38
1.14.2 The Rule From Now On: The App Asks, The Kernel Does . . . 38
Logical vs. Physical Protection . . . 39
1.14.3 The System-Call ABI and Services . . . 39
1.14.4 User Space Implementation: The Stubs . . . 40
1.14.5 The Kernel Entry Point: Why It Must Be Assembly . . . 41
1.14.6 Extracting the Service Number and Arguments . . . 41
1.14.7 The Dispatcher and Validating the Result Return . . . 41
Why Write to `frame[0]`? . . . 42
Try It: Your First System Call . . . 42
Finding the Service Number in GDB . . . 42
1.14.8 User Space Printing and Hardware Wait States . . . 42
1.14.9 The Kernel Rules: Where SVC Works . . . 43
1.14.10 Why the Kernel Never Waits . . . 43

1.14.11 User Space Library: Implementing `printf` 43
Handling Partial Writes and Waiting (`write_all`) 43
1.14.12 Implementing `sleep_ms` in User Space 44
1.14.13 The Final System Boot Flow 44
1.14.14 The User Application: `app_main` 45
1.14.15 A Debug Print Inside the Kernel 46
1.14.16 Making the Kernel Wait 46
1.14.17 Privilege and Layers in GDB 47
1.15 Where to Find the Code 48

Chapter 1

Devices, Interrupts, and System Calls

Week 3: Devices, interrupts and system calls

At the end of week 2 your program could boot on its own: your `Reset_Handler` set up the stack, copied `.data`, zeroed `.bss`, configured the CPU's fault handling and priorities, and called `main()`. It printed through semihosting, a trap to the debugger host, and it ended with a first taste of protection: `main` dropped privilege, and an `svc` instruction asked privileged code to print on its behalf.

This week you turn that program into something that looks like a small operating system kernel. You stop relying on the host to print, and talk to a real device, the board's serial port, by reading and writing its registers. You write a **device driver** for it, first in the simplest way (the CPU waits for the device), then in the efficient way (the device tells the CPU when it is ready, through an **interrupt**). You add a clock that ticks every 10 ms. And you finish by building a real **system-call interface**: an unprivileged application that prints, reads keys and sleeps, without ever touching a register itself, because every request goes through the kernel.

By the end of the week every layer between an unprivileged `printf` and the serial port's registers has been written by you, and has a name and a file.

The week is organized in three units, and these notes follow the same order:

1. **Devices and a polled driver.** The board's peripherals and how they are connected, memory-mapped I/O and `volatile`, the serial port (UART), what a driver is, and a `printf`-like function for the kernel.

2. **Calling convention and interrupts.** How C functions pass arguments and results (the AAPCS), what the hardware does on exception entry and return, the SysTick timer, the interrupt controller (NVIC), and an interrupt-driven transmitter with a ring buffer.

3. **System calls: the kernel boundary.** A system-call ABI built on `svc`, the kernel entry in assembly, a dispatch table of services, user-side stubs, a user `printf`, and two rules every kernel follows.

Two optional tutorials go with this week. The *C for device drivers* tutorial covers the C techniques used everywhere in this week's code (fixed-width types, bit operations, register `struct` overlays, variadic functions, ring buffers, function pointers)

with small programs you can run on your laptop. The *gdb debugging* tutorial goes further than the commands used in these notes: breakpoints, watchpoints, the call stack and debugging exceptions, all on this week's code.

Where you start and where you end

This week's code lives in two sibling folders, `week-03/base/` and `week-03/solution/`. Each has its own `Makefile` and builds on its own. You start from `base/` and, step by step, you turn it into the program in `solution/`.

The base is not a copy of week 2's result. It is a skeleton designed for this week, with the parts that are long to type and not this week's topic already in place:

- the full `Makefile`, which builds every source folder and has three targets: `run`, `debug`, and `run-trace` (the same as `run` plus a QEMU option you meet in Unit 1);
- the linker script `scripts/mps2_m3.ld`, unchanged from week 2;
- `include/cmsdk.h`, the complete register map of the board's devices, which you read and use instead of typing;
- `boot/startup.s`, with week 2's complete CPU setup and the 16 core slots of the vector table (the device slots are added in Unit 2);
- `kernel/faults.c`, where the four fault handlers are named, silent infinite loops;
- the headers that declare every function of the week (`drivers/uart.h`, `drivers/systick.h`, `kernel/irq.h`, `kernel/kprintf.h`, `kernel/syscall.h`, `lib/format.h`, `lib/printf.h`, `lib/syscalls.h`, `src/app.h`), so that files can call each other from the start;
- in `lib/format.c`, the two helpers that turn a number into text.

Every other source file exists only as a **marker file**: a header comment and one line starting with `/* CLASS:` that says what goes there. Each step in these notes says which marker line you replace with which code. The base's `src/main.c` is privileged and contains an empty `for ( ;; )` loop with three `CLASS:` markers. If you build and run the base, `make run` shows **nothing**: the first visible output of this week is the first byte you send through the serial port yourself.

The finished program in `solution/` prints, when you run it and press `a` and `b` on your keyboard after a second:

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
uptime: 1000 ms
key: a
key: b
uptime: 2000 ms
uptime: 3000 ms
```

The first two lines come from the privileged kernel. Everything after them comes from an unprivileged application that can only *ask* the kernel to do things.

Unless a path in these notes says otherwise, file paths are relative to `week-03/solution/`. The code is fully commented there, line by line, so you can always compare what you typed with the reference. A practical note: QEMU runs with its serial port on the terminal (`-serial stdio`, monitor disabled), so you stop it with `Ctrl-C`.

1.1 Unit 1: Devices and a Polled Driver

1.1.1 CMSDK and CMSIS: Hardware versus Software

In the ARM ecosystem, it is important to distinguish between two similarly named standards:

- **CMSDK (Cortex-M System Design Kit):** This is **hardware**. It comprises ready-made peripheral designs (e.g., APB UART, APB timer, AHB GPIO) that chip designers place alongside the Cortex-M CPU. The registers of these blocks behave identically across all chips that implement them.
- **CMSIS (Cortex Microcontroller Software Interface Standard):** This is **software**. It defines a standardized structure for C header files and helper functions (e.g., `NVIC_EnableIRQ()`, `UART0->DATA`) provided by chip vendors.

Our emulated board uses CMSDK peripherals. However, to understand the bare-metal mechanics, we avoid vendor CMSIS libraries. Instead, our register map (`include/cmsdk.h`) is hand-written in a *CMSIS style* to maintain standard naming conventions while retaining full control over the underlying implementation.

1.1.2 The Emulated Board: MPS2 with AN385

The target hardware is the **MPS2** ARM development board, configured with the **AN385** FPGA image. This image provides a Cortex-M3 CPU surrounded by CMSDK peripherals. QEMU emulates this configuration via the `mps2-an385` machine target.

The entire system is driven by a **25 MHz clock**. All time-dependent peripheral settings (such as UART baud rates and timer periods) are computed based on this central frequency.

The hardware peripherals are mapped to specific addresses within the Cortex-M peripheral memory region (`0x40000000` to `0x5FFFFFFF`):

Address	Device
`0x40000000 / 0x40001000 / 0x40002000`	TIMER0 / TIMER1 / dual timer
`0x40004000 / 0x40005000 / 0x40006000`	UART0 / UART1 / UART2
`0x40008000`	Watchdog
`0x40010000 ...`	GPIO0 ... (general-purpose pins)
`0x40028000`	FPGAIO (board LEDs and buttons)

1.1.3 Hardware Architecture: AHB vs. APB Buses

A bus is the physical set of wires carrying load and store operations between the CPU, memory, and devices. The Cortex-M3 utilizes two distinct buses:

- **AHB (Advanced High-performance Bus):** The main, pipelined bus capable of completing one transfer per clock cycle. High-speed components like RAM, code memory, and the GPIO controller reside here.
- **APB (Advanced Peripheral Bus):** A simpler, slower, and lower-power bus designed for rarely accessed peripheral registers (e.g., UARTs, Timers, Watchdog).

An **AHB-to-APB bridge** connects the two, translating fast AHB accesses into APB accesses. From a software perspective, this routing is transparent: a register is simply a memory address. However, accessing an APB register incurs a slight performance penalty (a few extra cycles) due to bridge latency.

1.1.4 The Board's Interrupt Lines

Peripherals notify the CPU of critical events (e.g., a byte arriving, or a timer reaching zero) by asserting a physical **Interrupt Request (IRQ)** line. In the Cortex-M architecture, device IRQ line n is serviced by vector table slot $16 + n$.

The primary interrupt map for the AN385 board is as follows:

- **IRQ 0 / 1:** UART0 Receive (RX) / Transmit (TX)
- **IRQ 8 / 9:** TIMER0 / TIMER1
- **IRQ 10:** Dual timer

The Watchdog timer bypasses standard IRQ lines and directly raises a **Non-Maskable Interrupt (NMI)** at vector slot 2. For the initial implementation phase, interrupts will remain disabled, and the CPU will rely only on **polling** (asking the devices in a loop whether they are ready).

1.1.5 Software Organization: `include/cmsdk.h`

To interact with the hardware, the provided `include/cmsdk.h` file describes each device as a C `struct` **laid over its registers**. The first field maps to the device's base address, and each subsequent `uint32_t` field sits exactly 4 bytes after the previous one.

For example, the serial port is defined as `CMSDK_UART_TypeDef`:

```
typedef struct
{
    volatile uint32_t DATA;       /* 0x00 read = received byte, write = by
    volatile uint32_t STATE;      /* 0x04 status flags */
    volatile uint32_t CTRL;       /* 0x08 control flags */
    volatile uint32_t INTSTATUS;  /* 0x0C interrupt status; write 1 to cle
    volatile uint32_t BAUDDIV;    /* 0x10 baud-rate divider */
} CMSDK_UART_TypeDef;

#define CMSDK_UART0    ( ( CMSDK_UART_TypeDef * ) 0x40004000UL )
```

`CMSDK_UART0` is not a variable; it is the raw memory address `0x40004000` cast to a pointer of this struct type. Therefore, evaluating `CMSDK_UART0->STATE` translates into a 32-bit memory load from address `0x40004004` (the base address plus the offset of the second field).

To ensure architectural safety:

- Every field is declared `volatile` (Section 1.1.10 explains why: it stops the compiler from optimizing away hardware reads and writes).
- The file defines specific bit names for each register (e.g., `UART_STATE_TXBF` as `1UL « 0`).
- A `_Static_assert` using `offsetof` is placed at the end of each struct definition to verify the field offsets at compile time, turning any layout typo into an immediate build error.

This single header provides the structured mappings for the SysTick, NVIC, SCB (System Control Block), timers, watchdog, GPIO, and FPGAIO.

1.1.6 The Board's Memory Map from QEMU

Instead of relying on the documentation alone, we can ask QEMU to list the devices it models. QEMU features a **monitor** console to control the emulator itself. By launching QEMU paused (`-S`) with the monitor enabled:

```
qemu-system-arm -M mps2-an385 -display none -S -monitor stdio -serial nu
```

We can issue the command `info mtree -f` to print every address range and the component answering there. Because the output is massive, a compact one-liner can filter for the `0x400xxxxx` peripheral region:

```
printf 'info mtree -f\nquit\n' | qemu-system-arm -M mps2-an385 -display n
```

This reveals the active memory map:

```
0000000040000000-0000000040000fff (prio 0, i/o): cmsdk-apb-timer
0000000040002000-0000000040002fff (prio 0, i/o): cmsdk-apb-dualtimer
0000000040004000-0000000040004fff (prio 0, i/o): uart
0000000040008000-0000000040008fff (prio 0, i/o): cmsdk-apb-watchdog
0000000040010000-0000000040010fff (prio -1000, i/o): cmsdk-ahb-gpio
0000000040020000-0000000040020fff (prio 0, i/o): pl022
0000000040022000-0000000040022fff (prio 0, i/o): arm_sbcon_i2c
0000000040028000-0000000040028fff (prio 0, i/o): mps2-fpgaio
```

Interpreting the Output: Real Models vs. Placeholders

The `prio` field dictates how QEMU handles the memory range:

- `prio 0` **(Real Models):** Devices like `uart`, `cmsdk-apb-timer`, and `mps2-fpgaio` are fully emulated. QEMU models five UARTs total; `UART0` at `0x40004000` is the one actively connected to your terminal via the `-serial stdio` flag. Two additional serial interfaces appear here: `pl022` (SPI) and `arm_sbcon_i2c` (I2C).
- `prio -1000` **(Placeholders):** Devices like `cmsdk-ahb-gpio` are "unimplemented devices". The emulator accepts memory accesses to these addresses without throwing a fault, but it silently ignores all writes and returns 0 on all reads.

Touching a Device That Is Not There

QEMU can log every time the software attempts to access an unimplemented placeholder device by using the `-d unimp` flag (which is integrated into the project's `make run-trace` command).

If you build the project, pause the emulator at reset (`-S`), and attach GDB, you can manually poke the system bus:

```
(gdb) set {unsigned int}0x40010010 = 1        # GPIO0 OUTENABLESET
(gdb) x/wx 0x40010010
(gdb) set {unsigned int}0x40028000 = 3        # FPGAIO LED register
(gdb) x/wx 0x40028000
```

In the QEMU terminal, the GPIO operations (which target a placeholder) trigger warning logs, while the read returns `0`:

```
cmsdk-ahb-gpio: unimplemented device write (size 4, offset 0x010, value 0x00000001)
cmsdk-ahb-gpio: unimplemented device read  (size 4, offset 0x010)
```

Conversely, the FPGAIO register (a real model) accepts the write, retains the value `0x00000003`, and logs nothing.

In summary, a peripheral access in the emulator can conclude in three ways, none of which triggers a hardware fault, allowing the software to silently carry on:

- **Real Model** (e.g., UART0, SysTick): The device functions normally; nothing is logged by `-d unimp`.
- **Placeholder for a Device** (e.g., GPIO0): The access is safely ignored or zeroed; QEMU logs the specific device name (`cmsdk-ahb-gpio`).
- **No Device At All** (e.g., `0x40080000`): The access is ignored; QEMU logs a generic region-wide placeholder name such as `RESERVED 4`.

If your code appears to do nothing, `make run-trace` can reveal whether the software is writing to a non-existent device. Note also the distinction between **core peripherals** (like SysTick and the NVIC, which the ARMv7-M architecture guarantees will exist on every Cortex-M) and **board peripherals** (like UART and GPIO, whose presence and placement depend on the board designer and emulator configuration).

1.1.7 The General I/O Landscape

While this course focuses primarily on driving the UART, a modern microcontroller interacts with the outside world through a variety of standardized hardware interfaces:

Complex interfaces like SPI and I2C are programmed using the same memory-mapped register concepts that we will apply to the UART, simply requiring more state-machine steps per transfer.

Interface	Wires	Mechanism	Typical Use
GPIO	1 per pin	Programmable as input (read logic level) or output (drive 0 or 1).	Buttons, LEDs
UART	2 (TX, RX)	Asynchronous: no shared clock. Devices must agree on a transmission speed (baud rate).	Console, PC link
SPI	CLK, Data In, Data Out, CS	Synchronous: controller drives a shared clock. Uses Chip-Select (CS) to pick a specific device.	Flash memory, displays
I2C	2 (CLK, DATA)	Synchronous: shared bus. Controller routes data using 7-bit device addresses.	Sensors
ADC / DAC	1 per channel	Converts an analog voltage to a digital number (ADC) or vice versa (DAC).	Temperature, Audio

1.1.8 DMA: Moving Data Without the CPU

In all the code of this week, the CPU moves every byte itself: it reads a register, stores the value in memory, and repeats. For a large transfer, such as a flash sector or an audio buffer, the CPU is busy for the whole time.

A **DMA controller** (Direct Memory Access) is a device that copies data by itself. You program a source address, a destination address and a count, and start it. The DMA controller then uses the bus while the CPU runs other code, and signals the CPU when it is done, usually with an interrupt. Typical uses are peripheral to memory (bytes received by a UART, samples from an ADC) and memory to peripheral (data to send on SPI). The cost is that the CPU and the DMA controller share the bus, and the software must not touch a buffer while the DMA controller is still filling it. QEMU's AN385 has no DMA controller, which is why the CPU moves every byte here.

1.1.9 Memory-Mapped I/O: Loads and Stores with Side Effects

Every peripheral register has a fixed address, and you talk to a device with ordinary load and store instructions at that address. This is called **memory-mapped I/O** (MMIO). Written without the `struct`, sending a byte looks like this:

```
#define UART0_DATA (*(volatile uint32_t *)0x40004000u)
UART0_DATA = 'A';        /* a store to 0x40004000: the UART sends 'A' */
```

`CMSDK_UART0->DATA = 'A';` is exactly the same store, written through the `struct`. It looks like an assignment to a variable, but a device register is **not** RAM, and four differences matter:

- a **read** can have an effect: reading the UART's `DATA` register takes the received byte out of the device;

- a **write** can start an action: writing `DATA` starts sending a byte;
- a register can **change without any store** from the CPU: a status bit goes to 1 when a byte arrives;
- the value you read back may differ from the value you wrote: some bits are read-only, others clear themselves.

The device's manual says what each read and each write does, and your code must follow it exactly.

1.1.10 The `volatile` Keyword

The compiler assumes that it is working with ordinary memory, where a value changes only when **your program** stores to it. With optimization turned on, it keeps values in CPU registers and removes reads it believes are repeated. Consider the loop that waits for the UART to have room:

```
while ( UART0_STATE & UART_STATE_TXBF )    /* wait until the UART has room
{
}
```

Without `volatile`, the compiler sees a loop that never writes `UART0_STATE`. In its model the value "cannot change", so it reads it once and, if the bit was 1, spins forever without looking again.

The keyword `volatile` tells the compiler: **every read and every write in the source must really happen**, in the order written with respect to other `volatile` accesses. You used it in week 2's divide-by-zero demo, to stop the compiler from computing the result at build time. Two things `volatile` does **not** do: it does not make an access atomic (indivisible), and it does not protect data shared with other code. It only forbids the compiler from removing or merging accesses.

Try It: `volatile` at `-O2`

You can watch the compiler delete a hardware read. In a folder outside `base/` (for example `/tmp/vol`), create `poll.c`:

```
#include <stdint.h>

#ifndef QUAL
#define QUAL     /* empty: a plain pointer */
#endif

/* UART0 STATE, bit 0 = TXBF */
#define UART0_STATE (*(QUAL uint32_t *)0x40004004u)

void wait_tx_room(void)
{
    while (UART0_STATE & 1u)
    {
        /* wait until TXBF is 0 */
    }
```

}

Compile it twice with optimization level `-O2`, once plain and once with `-DQUAL=volatile` (which defines `QUAL` as `volatile`), then disassemble both object files:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -O2 -c poll.c -o plain.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -O2 -DQUAL=volatile -c poll.c -
arm-none-eabi-objdump -d plain.o
arm-none-eabi-objdump -d vol.o
```

The question is: in which version does the loop jump back to the `ldr`, the instruction that reads `STATE`? In `plain.o` the read happens once, and the "loop" is a branch to itself:

```
0: mov.w r3, #0x40004000   /* r3 = 0x40004000                          */
4: ldr   r3, [r3, #4]      /* r3 = STATE   (read ONCE)                  */
6: lsls  r3, r3, #31       /* shift bit 0 into the sign: N flag = TXBF */
8: bmi.n c                 /* if (TXBF) goto c                          */
a: bx    lr                /* return                                    */
c: b.n   c                 /* c: goto c  -> for (;;) { }, never reads again */
```

In `vol.o` the loop goes back to the `ldr` and reads `STATE` on every turn:

```
0: mov.w r2, #0x40004000   /* r2 = 0x40004000                 */
4: ldr   r3, [r2, #4]      /* r3 = STATE   (read every turn)  */
6: lsls  r3, r3, #31       /* N flag = TXBF                   */
8: bmi.n 4                 /* if (TXBF) goto 4: read again    */
a: bx    lr                /* return                          */
```

At `-O0`, the level this course's `Makefile` uses, even the plain version happens to re-read the register, so the bug stays hidden until someone turns optimization on. The rule is simple: always access device registers through `volatile`.

1.2 Read-Modify-Write Needs Care

When configuring a device, you often need to change a single bit while leaving the rest of the register intact. In C, this is commonly written using combined assignment operators, like the bitwise OR assignment:

```
CMSDK_GPIO0->DATAOUT |= (1u << 3); /* set pin 3: read, OR the bit, write
```

While this looks like a single line of C code, at the hardware level, it compiles into three distinct operations:

1. **Read:** Load the current value of the register from memory into a CPU register.
2. **Modify:** Perform the bitwise OR operation.
3. **Write:** Store the modified value back to the hardware memory address.

These three operations are **not atomic**. This introduces several hardware pitfalls:

- **Concurrency conflicts:** If an interrupt fires or the hardware autonomously changes a different bit in the register between your read and your write, your subsequent write will forcefully overwrite the hardware's changes with outdated data.
- **Write-1-to-clear (W1C) bits:** Certain status registers clear their flags when you write a 1 to them, leaving them alone if you write a 0. Using a read-modify-write pattern on a W1C register reads back all currently set flags (as 1s) and writes them straight back, accidentally clearing every pending interrupt flag.
- **Read $\neq$ Written:** Some registers return different values when read compared to what was written. Reading and writing back such a register can unintentionally scramble bits you did not mean to touch.

For more on bit operations (set, clear, toggle, test, multi-bit fields) see the *C for device drivers* tutorial. In particular, note why we write `1u << n` rather than `1 << n`: the literal `1` is a signed `int`, so shifting it into bit 31 is undefined behavior, while `1u` is unsigned.

1.2.1 Safer Hardware Patterns

To avoid these pitfalls:

- **Set / Clear Registers:** Many modern peripherals provide dedicated registers strictly for setting or clearing bits (e.g., `OUTENABLESET`). Writing `CMSDK_GPIO0->OUTENABLESET = 1u << 3;` is a single write operation; the hardware ensures that only the bits set to 1 are modified.
- **W1C Registers:** Always use a plain assignment (`=`) containing only the specific bit you want to clear. Never use `|=`.
- **Unavoidable Read-Modify-Write:** If a read-modify-write is strictly required, make sure nothing else can touch the register in the meantime (as for Week 2's CCR, where the `STKALIGN` bit had to stay 1). We will learn to handle this gracefully later in the course.

1.3 The Device Protocol and the Driver Concept

Interacting with a hardware device requires following a strict state machine protocol dictated by the device's manual. A device is not a single variable, but a **protocol** expressed through its registers, with a fixed order of steps:

```
configure --> enable --> check status --> move data --> acknowledge
(BAUDDIV)     (CTRL)     (STATE)          (DATA)        (INTSTATUS)
```

- You **configure** before you **enable**: set the speed before switching the transmitter on.
- You **check the status** before **moving data**: write `DATA` only when the device has room, and read it only when a byte is there.

- You **acknowledge** an event by clearing its flag, so that the device can report the next one.

A **device driver** is the code that implements this protocol once, offering the rest of the system **functions with a meaning** ("send this byte") instead of addresses and bits. It acts as an **API boundary**: callers include the driver's header (`drivers/uart.h`) and never `cmsdk.h`, keeping hardware addresses abstracted.

We will build a UART driver to replace Week 2's artificial semihosting trick with real hardware communication. The driver API starts small:

```
void uart_init(void);                  /* Set up UART0; called once at boot
void uart_putc_polled(char c);         /* Send one byte; waits if buffer is
int uart_getc_polled(void);            /* Get a byte, or -1 if none is waiti
```

Polled means the driver reads the hardware status register in a loop (for sending) or once per call (for receiving) to check for readiness, without using interrupts.

1.3.1 The CMSDK UART Registers

UART stands for Universal Asynchronous Receiver-Transmitter. The CMSDK UART has five registers:

Offset	Register	Access	Bits
`0x00`	`DATA`	read / write	write: byte to send; read: received byte (`RXBF` goes back to 0)
`0x04`	`STATE`	read; bits 2–3 W1C	0 **TXBF**: send buffer full; 1 **RXBF**: byte waiting; 2 `TXOR`, 3 `RXOR`: overruns
`0x08`	`CTRL`	read / write	0 **TXEN**; 1 **RXEN**; 2 `TXIRQEN`; 3 `RXIRQEN`; 4–5 overrun IRQ enables
`0x0C`	`INTSTATUS`	read / W1C	0 TX, 1 RX, 2 TXOR, 3 RXOR: interrupt pending; write 1 to clear
`0x10`	`BAUDDIV`	read / write	clock divided by the baud rate, at least 16

The UART holds **one byte** in each direction. When `TXBF` is 1, the last byte you wrote is still going out, and a byte written now is lost (transmit overrun, `TXOR`). When `RXBF` is 1, a received byte is waiting, and if another byte arrives before you read it, it is lost (receive overrun, `RXOR`). After reset, `CTRL` is 0: the transmitter and receiver are **off**, and the UART ignores anything written to `DATA`.

Configuring the Baud Rate

Because UART is asynchronous, both devices must agree on a transmission speed (baud rate). You divide your CPU's clock frequency by your target baud rate. If our CPU runs at 25 MHz and we want 115,200 baud:

$$\text{BAUDDIV} = \frac{25,000,000}{115,200} \approx 217$$

Writing 217 into `BAUDDIV` configures the hardware correctly.

1.4 Implementing the Device Driver

1.4.1 How Bigger OSes Generalize a Driver

If a system has many character devices, the OS needs a generic way to talk to them. Operating systems solve this by defining a `struct` composed of function pointers:

```
struct char_driver
{
    void (*init)(void);          /* bring the device up */
    int  (*read)(void);          /* read one byte, or -1 if none */
    void (*write)(char c);       /* send one byte */
};

static const struct char_driver uart0_driver = {
    uart_init,
    uart_getc_polled,
    uart_putc_polled
};
```

This allows generic code to interact with any device (e.g., `dev->write('h');`) without knowing its specifics. Linux does exactly this with `struct file_operations`. This week's kernel has a single console and calls the UART functions directly, so you will not write this `struct` yet.

1.4.2 Implementing and Using the UART Driver

In `drivers/uart.c`, paste the following right **above** the marker line `/* CLASS: uart_init, polled putc/getc, ...` (keep the marker for Unit 2).

```
#include "drivers/uart.h"
#include "cmsdk.h"

void uart_init(void)
{
    /* configure: speed (QEMU ignores it) */
    CMSDK_UART0->BAUDDIV = 16;
    /* enable: transmitter and receiver */
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;
}

void uart_putc_polled(char c)
{
    /* check status: wait for room */
    while (CMSDK_UART0->STATE & UART_STATE_TXBF)
    {
    }
    /* move data: send the byte */
    CMSDK_UART0->DATA = (uint32_t)(unsigned char)c;
}
```

Following the protocol, `uart_init` configures and enables the hardware. Because the UART is an external physical device, initialization happens only **once** at boot. We combine two bitmasks with a bitwise OR to enable both directions simultaneously.

In `uart_putc_polled`, if the `TXBF` bit is 1, the CPU loops waiting; once free, the byte is stored into `DATA`. **Why the double cast?** On Arm, `char` is unsigned, but on many compilers it is signed. A signed `char` holding `0xE9` is -23, which converts to `uint32_t` as `0xFFFFFFE9`. Casting through `unsigned char` ensures it becomes `0x000000E9`, extracting exactly the byte.

Testing the Driver In `src/main.c`, add `#include "drivers/uart.h"` above `main`, and right after the marker `/* CLASS: bring up the UART here */` add:

```
#include "drivers/uart.h"

int main(void)
{
    /* CLASS: bring up the UART here */
    uart_init();
    uart_putc_polled('O');
    uart_putc_polled('K');
    uart_putc_polled('\n');
    for (;;) { }
}
```

`make run` prints `OK`: the first real hardware output of our OS. If you comment out `uart_init()`, `CTRL` remains 0 and the UART drops every byte. Running QEMU with `-d guest_errors` reports:

```
CMSDK APB UART: transmit data write with Tx disabled
```

This enforces the rule "enable before moving data".

1.4.3 Try It: A Polled Echo

In `drivers/uart.c`, right after `uart_putc_polled`, add the reading function. It returns `-1` immediately if no character is waiting:

```
int uart_getc_polled(void)
{
    /* check status: nothing received yet */
    if ((CMSDK_UART0->STATE & UART_STATE_RXBF) == 0)
    {
        return -1;
    }
    /* move data: reading DATA takes the byte */
    return (int)(CMSDK_UART0->DATA & 0xFF);
}
```

In `src/main.c`, replace the empty `for (;;) { }` at the end of `main` with an echo loop:

```
    for (;;)
    {
        int c = uart_getc_polled();
        if (c >= 0)              /* -1: no key */
        {
            uart_putc_polled((char)c);
        }
    }
```

Run `make run` and type `hello`. With `-nographic`, the terminal does not show what you type: every letter you see was **sent back by your program**. The Enter key sends a carriage return (`\r`), jumping the cursor back to the start of the line. The CPU spins in this loop millions of times per second: this is the price of polling.

1.5 Variadic Functions and <stdarg.h>

Currently, our driver can only print single characters. To build a true operating system, we need a `printf`-like function capable of formatting complex strings and numbers (e.g., `%d`, `%s`, `%x`). We will call our kernel-level version `kprintf`.

To build `kprintf`, we face two tasks:

1. String formatting (parsing the `%d` markers and converting integers to ASCII characters).
2. Sending the resulting characters to the UART.

In C, a function that accepts an arbitrary number of arguments (like `printf`) is called a **variadic function**. It is declared using an ellipsis (`...`):

```
void kprintf(const char *fmt, ...);
```

`<stdarg.h>` is one of the headers a freestanding build still gives you, as you saw in Week 2: it contains only types and compiler built-ins, no library code. It provides the tools to walk through the extra arguments: `va_list`, `va_start`, `va_arg` and `va_end`.

```
#include <stdarg.h>

void kprintf(const char *fmt, ...)
{
    va_list ap;                  /* a cursor over the extra arguments
*/
    va_start(ap, fmt);           /* start right after the last named one
    int n = va_arg(ap, int);     /* take the next argument, as an int
*/
    va_end(ap);                  /* done
*/
}
```

A variadic function cannot see how many arguments it received, nor their types. The **format string** tells it: `%d` means "take an `int`". If the type in the format and the type actually passed disagree, the behavior is undefined. Arguments passed

through ... are also **promoted**: a char arrives as an int, so %c must read it with va_arg(ap, int).

By convention, a function that receives a va_list instead of ... has a name starting with v, such as vprintf in the C library. The *C for device drivers* tutorial (Section 4) shows variadic functions, promotions and a tiny format dispatcher you can run on your laptop.

1.5.1 Separating Formatting from Output (vformat)

Turning "x=%d" and 42 into x=42 is the same work wherever the characters end up. So lib/format.h (given) separates **what** to print from **where** it goes:

To maintain a clean operating system architecture, we separate the formatting engine from the hardware driver using a function called vformat (implemented in lib/format.c).

```
void vformat(void (*out)(char c, void *ctx), void *ctx,
             const char *fmt, va_list ap);
```

out is a **function pointer**: vformat calls out(c, ctx) once for every character it produces. ctx is passed to out untouched, and holds whatever state the output needs (for example a buffer and a position in it).

kprintf is then vformat with an out that calls uart_putc_polled, and a ctx of 0 because it needs no state.

Another destination needs only another small out function: the formatting is written and tested once.

vformat supports %c, %s, %d (signed decimal), %u (unsigned decimal), %x (hexadecimal, lowercase, without 0x) and %% (a percent sign).

There is no width and no precision, and an unknown conversion such as %q is printed as written.

The two helpers that turn a number into text, utoa_dec and utoa_hex, are already in lib/format.c in the base.

1.5.2 Writing vformat

In lib/format.c, replace the marker line /* CLASS: vformat (the % dispatch: %c %s %d %u %x %%) goes here. */ with a helper and the function itself.

The helper, put_string, sends every character of a string to out:

```
static void put_string(void (*out)(char c, void *ctx), void *ctx,
                       const char *s)
{
    while (*s != '\0')
    {
        out(*s, ctx);
        s++;
    }
}
```

vformat walks through the format string. An ordinary character is passed on; a % makes it look at the next letter and take one argument of the right type:

```
void vformat(void (*out)(char c, void *ctx), void *ctx,
             const char *fmt, va_list ap)
{
    char num[12];    /* longest number + '\0' */

    while (*fmt != '\0')
    {
        if (*fmt != '%')                /* ordinary character: pass it on
        {
            out(*fmt, ctx);
            fmt++;
            continue;
        }
        fmt++;                          /* a '%': look at the next letter
        switch (*fmt)
        {
        case 'c':
            out((char)va_arg(ap, int), ctx);   /* a char arrives as an in
            break;
        case 's':
        {
            const char *s = va_arg(ap, const char *);
            put_string(out, ctx, s != 0 ? s : "(null)");  /* no crash on
            break;
        }
        case 'd':
        {
            int v = va_arg(ap, int);
            uint32_t mag;
            if (v < 0) { out('-', ctx); mag = 0u - (uint32_t)v; }
            else       { mag = (uint32_t)v; }
            utoa_dec(mag, num);
            put_string(out, ctx, num);
            break;
        }
        case 'u': utoa_dec(va_arg(ap, uint32_t), num); put_string(out, ct
        case 'x': utoa_hex(va_arg(ap, uint32_t), num); put_string(out, ct
        case '%': out('%', ctx); break;
        case '\0': out('%', ctx); return;                    /* string ende
        default:  out('%', ctx); out(*fmt, ctx); break;      /* unknown: as
        }
        fmt++;                          /* past the conversion letter */
    }
}
```

(The solution file spreads the same code over more lines, with a comment on each.)

Why `0u - (uint32_t) v` and not `-v`? Because the most negative `int`, $-2\,147\,483\,648$ (`INT_MIN`), has no positive `int` counterpart, so `-v` overflows, which is undefined behavior in C.

Unsigned subtraction is always well defined, and gives $2\,147\,483\,648$. After this step `make` must build with no errors.

1.6 The Kernel Print Function (`kprintf`)

With the formatting engine and the UART driver both ready, we can combine them to create a real print function. We name it `kprintf` (where the "k" stands for kernel).

1.6.1 Kernel Mode vs. User Mode Printing

Naming this function `kprintf` highlights a fundamental operating system concept. In a real OS (like Linux), there is a strict separation between how the kernel prints messages and how a user application prints messages:

- **User-Level `printf`:** A user application runs in a restricted user mode. It does not know that a UART even exists, nor does it have the hardware privileges to touch it. When a user program calls `printf`, it triggers a `system call`, asking the OS to print on its behalf. The OS then securely routes that request to the appropriate device driver.
- **Kernel-Level `kprintf`:** The OS kernel itself runs in privileged (supervisor) mode. It inherently knows about the hardware and the I/O devices. Therefore, `kprintf` bypasses system calls entirely and talks directly to the hardware driver. This is not only a design choice: the kernel *cannot* use `printf` for its own messages, because an `svc` issued from the SVCall handler escalates to a HardFault.

Any operating system that implements hardware protection features both a user-side print function and a kernel-side print function. We are currently inside the operating system layer, so we implement `kprintf` first. (The more complex user-level system call mechanism will be implemented later).

1.6.2 Implementing `kprintf`

In `kernel/kprintf.c`, we write a small wrapper function that connects `vformat` to our polled UART driver.

First, we define the callback function that `vformat` expects. This simply takes a character and hands it to the driver:

```
#include "kernel/kprintf.h"
#include "lib/format.h"
#include "drivers/uart.h"

/* vformat's output: one char to the UART */
static void kprintf_out( char c, void *ctx ){
```

```
    (void) ctx;      /* not used */
    uart_putc_polled( c );
}
```

Then, we implement the variadic `kprintf` function itself. It collects its extra arguments into a `va_list` and hands everything to `vformat`, with `kprintf_out` as the destination:

```
void kprintf( const char *fmt, ... )
{
    va_list ap;
    va_start( ap, fmt );
    vformat( kprintf_out, 0, fmt, ap );
    va_end( ap );
}
```

Because `kprintf` relies on our polled UART driver, it operates entirely without hardware interrupts. This is a massive architectural advantage: it means `kprintf` can be safely executed anywhere in the kernel, even inside a catastrophic `HardFault` handler where interrupts are strictly disabled.

To test every conversion, we can add a test line in `src/main.c`:

```
kprintf( "c=%c s=%s d=%d u=%u x=%x pct=%%\n", 'A', "hi", -42, 4000000000u
```

This outputs `c=A s=hi d=-42 u=4000000000 x=beef pct=%`. Notice that `%x` prints no `0x` prefix; if you want one, you must write it explicitly in the format string (e.g., `"0x%x"`).

You can follow a call by hand to check the `vformat` logic. For `kprintf("d=%d\n", -42)`:

- The loop passes `d` and `=` to `kprintf_out`.
- It meets `%`, moves to `d`, takes an `int` (-42) with `va_arg`.
- It sends `-`, computes the magnitude 42, turns it into the text `"42"` with `utoa_dec`, and sends `4` and `2`.
- It moves past the `d`, passes `\n`, and stops at the terminating zero.

This results in exactly six calls to `kprintf_out`, pushing six bytes sequentially to the UART.

1.7 Upgrading the Kernel: Fault Handlers That Print

The base's fault handlers are silent infinite loops: if a hardware fault happens, the program simply stops answering. Now that the kernel can print using our robust, polled `kprintf`, the handlers can explicitly state what happened.

1.7.1 The Status Registers (HFSR and CFSR)

Two specific registers of the System Control Block (SCB) tell you why a fault happened:

- **CFSR (Configurable Fault Status Register):** Located at `0xE000ED28`, this register has one specific bit for each possible cause of a `MemManage` fault, a `BusFault`, or a `UsageFault`.
- **HFSR (HardFault Status Register):** Located at `0xE000ED2C`, this indicates exactly why a `HardFault` was escalated. For example, bit 30 (`FORCED`, `0x40000000`) means that a fault was escalated to HardFault.

1.7.2 Implementing the New Handlers

In `kernel/faults.c`, we include the necessary headers and replace the silent loops with explicit print statements before halting:

```
#include "cmsdk.h"
#include "kernel/kprintf.h"

void HardFault_Handler(void) {
    kprintf( "HardFault: HFSR=0x%x\n", SCB->HFSR );
    for(;;){}
}
void MemManage_Handler(void) {
    kprintf( "MemManage: CFSR=0x%x\n", SCB->CFSR );
    for(;;){}
}
void BusFault_Handler(void) {
    kprintf( "BusFault: CFSR=0x%x\n", SCB->CFSR );
    for(;;){}
}
void UsageFault_Handler(void) {
    kprintf( "UsageFault: CFSR=0x%x\n", SCB->CFSR );
    for(;;){}
}
```

1.7.3 Try It: Catching a Divide-By-Zero

To test this new capability, we can intentionally crash the OS. Week 2's `Reset_Handler` set the `DIV_0_TRP` bit in the CCR, meaning an integer division by zero strictly triggers a hardware fault.

Add the following temporary code to `src/main.c`:

```
volatile unsigned zero = 0;      /* read at run time */
kprintf( "%u\n", 10u / zero );   /* should fault     */
```

The `volatile` keyword is crucial here; it forces the compiler to perform the math at runtime rather than computing the result at build time.

Running `make run` yields:

```
UsageFault: CFSR=0x2000000
```

The value `0x2000000` indicates that bit 25 is set, which perfectly corresponds to `SCB_CFSR_DIVBYZERO` in `include/cmsdk.h`. The CPU instantly fired a `UsageFault`,

the handler used the polled `kprintf` to push the error string over the UART, and then the system cleanly locked up in the infinite loop.

1.7.4 Practical Note: Project Navigation and Exam Expectations

As we build these low-level layers, the codebase is becoming increasingly complex. Files are spread across different directories (`drivers/`, `kernel/`, `lib/`, `src/`), each serving a distinct architectural role.

From an academic and practical standpoint, it is critical to understand how to navigate this project structure. During an exam (or in a real-world debugging scenario), you will not be asked to write a full operating system from scratch in 90 minutes. Instead, you will typically be given an existing, structured project and asked to implement a specific function, fix a bug, or integrate a new hardware feature. Mastering how to read the existing code, locate the correct files, and understand the flow of execution is just as important as understanding the hardware registers themselves.

1.8 Unit 2: Calling Convention and Interrupts

Up to this point, our entire OS relies on polling. The `uart_putc_polled` driver explicitly reads the hardware `STATE` register in an infinite loop until the transmitter has room.

1.8.1 Polling versus Interrupts

While polling is simple, it is highly inefficient for a general-purpose operating system. The alternative is an **interrupt**: the device actively raises a physical signal when it requires attention, the CPU instantly pauses its current task, runs a handler (an exception), and then resumes its previous work.

	Polling	**Interrupts**
CPU time	Wasted in a waiting loop, even when nothing is happening.	Spent only when an event actually happens (entry, handler execution, exit).
Latency	Up to one full trip around the while loop.	The exception entry time ($\sim$12 cycles on Cortex-M3), assuming nothing more urgent is running.
Complexity	Extremely simple; one loop.	Complex: requires handlers, shared data protection, and priority management.
CPU state	Constantly busy spinning.	Free to run other application work, or safely asleep.

To contextualize the performance penalty: a UART transmitting 10 bits at 115,200 baud requires $\sim$87 μs per byte. At our board's 25 MHz clock, that equates to **2,170 cycles**. With polling, all 2,170 cycles are wasted waiting.

However, polling is necessary for events that are certain and very close in time, or when interrupts cannot fire. This is why `kprintf` remains polled: it must guarantee output even when the system crashes inside a fault handler. Interrupts are suited for rare events (a key press) or lengthy data transfers.

1.8.2 The AAPCS: How C Calls a Function

To write assembly code that C can call, both must adhere to the same rules for passing arguments and results. For ARM, these are defined in the **AAPCS** (Procedure Call Standard for the Arm Architecture).

The standard defines strict roles for the core registers:

- `r0-r3`: Pass the first four arguments. `r0` also carries the function's return value. These are **Caller-Saved** (scratch) registers: a function can overwrite them freely.
- `r4-r11`: Used for the function's own long-lived local variables. These are **Callee-Saved** registers: if a function uses them, it must restore their original values before returning (e.g., `push {r4, lr}` and `pop {r4, pc}`).
- `r12`: A scratch register, strictly **Caller-Saved**.
- `lr` **(Link Register):** Holds the return address written by the `bl` instruction. It is **Caller-Saved**, so a function that calls another must save it first.
- `sp` **(Stack Pointer):** Must remain a multiple of 8 at every function boundary.

1.8.3 Writing Assembly Callable from C: The CPU's Interrupt State

Before handling hardware interrupts, we must manage the CPU's global ability to listen to them. The Cortex-M CPU has a 1-bit hardware register called **PRIMASK**. When `PRIMASK` is 1, all configurable interrupts are masked (disabled); when 0, they are allowed.

Because `PRIMASK` is a core register, C cannot access it directly. It requires dedicated assembly instructions:

- `mrs r0, primask`: Move to Register from Special register (reads `PRIMASK` into `r0`).
- `msr primask, r0`: Move to Special register from Register (writes `r0` into `PRIMASK`).
- `cpsid i` / `cpsie i`: Change Processor State (Disable / Enable Interrupts).

The Nesting Problem

A naive approach to critical sections is executing `cpsid i` at the start and `cpsie i` at the end. However, if a function that masks interrupts calls another function that also masks them, the inner function's `cpsie i` would unmask interrupts too early upon exit, destroying the outer function's critical section.

To solve this safely, we design an API that saves and restores the state:

```
uint32_t irq_save( void );          /* mask interrupts, return the old PR
void irq_restore( uint32_t old );   /* restore PRIMASK to the old state *
```

Nested calls will correctly maintain the masked state until the outermost function restores the original unmasked value.

Implementing `irq_save` and `irq_restore` in Assembly

We implement these in `kernel/irq.s`. Following the AAPCS, `irq_save` leaves the resulting 32-bit `PRIMASK` state in `r0` before branching back. `irq_restore` receives its argument in `r0`. Both only use `r0` and `lr` (Caller-Saved), so they do not need to save Callee-Saved registers.

```
    .syntax unified                 /* modern ARM assembly syntax */
    .cpu cortex-m3                  /* only instructions this CPU has */
    .thumb                          /* Thumb code */
    .section .text,"ax",%progbits

/* uint32_t irq_save(void) */
    .global irq_save                /* visible to C */
    .type irq_save, %function       /* it is code */
    .thumb_func                     /* Thumb: bit 0 set in its address */
irq_save:
    mrs r0, primask                 /* old = PRIMASK; r0 = return value */
    cpsid i                         /* PRIMASK = 1: interrupts masked */
    bx lr                           /* return old (in r0) */
    .size irq_save, . - irq_save

/* void irq_restore(uint32_t old) */
    .global irq_restore             /* visible to C */
    .type irq_restore, %function
    .thumb_func
irq_restore:
    msr primask, r0                 /* PRIMASK = old (first argument, r0) */
    bx lr                           /* return */
    .size irq_restore, . - irq_restore
```

Code Breakdown:

- `.global`, `.type`, `.thumb_func`, `.size`: Directives defining visibility, executable Thumb status, and size.
- `mrs r0, primask`: Copies `PRIMASK` into `r0`, setting up the return value.
- `cpsid i`: Sets `PRIMASK` to 1, immediately masking configurable interrupts.
- `msr primask, r0`: Moves the saved state (`r0`) back into `PRIMASK`. If the state was 0, interrupts turn back on; if 1, they remain disabled.
- `bx lr`: Branches to the Link Register, returning execution to C.

Testing the API in C

Define the prototypes in `kernel/irq.h`:

```
#ifndef IRQ_H
#define IRQ_H
#include <stdint.h>

uint32_t irq_save(void);
void irq_restore(uint32_t state);

#endif
```

In `main.c`, include `kernel/irq.h` and test the API:

```
#include "kernel/irq.h"

/* Inside main() */
uint32_t old_state = irq_save();        /* Save current state and disable i

/* Right here, interrupts are masked. */

irq_restore(old_state);                 /* Restore the state back to what w

kprintf("PRIMASK before irq_save was: %u\n", old_state);
```

Executing this prints: `PRIMASK before irq_save was: 0`. This confirms the CPU started with interrupts enabled, and our assembly safely disabled and cleanly restored that state.

Watching the AAPCS Across the Call (GDB)

We can observe the AAPCS mechanics in GDB:

```
(gdb) break irq_save
(gdb) continue
(gdb) p/x $lr
(gdb) info registers primask
(gdb) finish
(gdb) info registers r0 primask
```

At the breakpoint, `lr` holds the return address written by `bl` (the address is odd, e.g., `0x4a9`, because bit 0 marks Thumb code). `PRIMASK` starts as `0`. After `finish`, `r0` contains `0x0` (returned old value) and `PRIMASK` is `1` (masked). After `irq_restore(old_state)`, `PRIMASK` returns to `0`. Data traveled exactly as the AAPCS specifies via `r0`.

1.9 ARM Exception Entry and Return Mechanics

While we can wrap many features in simple C functions, handling exceptions requires a deep understanding of what the Cortex-M hardware does autonomously. When a hardware interrupt fires, the CPU abruptly pauses the current execution to jump to

an Exception Handler. Before executing the handler's first instruction, the hardware performs three critical operations:

1. It pushes exactly 8 words (the exception frame) onto the current stack.
2. It writes a special signature value, called `EXC_RETURN`, into the Link Register (`lr`).
3. It reads the specific handler's memory address from the vector table and branches to it, automatically elevating to privileged Handler mode.

1.9.1 The Automatic Hardware State Save (Stacking)

The 8 words pushed to the stack form the stacked frame. Because the stack grows downwards, the hardware creates the following memory layout relative to the new Stack Pointer (`sp`):

- `sp + 28`: `xPSR` (Status flags, Thumb bit)
- `sp + 24`: `PC` (Program Counter: where the interrupted code will resume)
- `sp + 20`: `LR` (The interrupted code's Link Register)
- `sp + 16`: `r12`
- `sp + 12`: `r3`
- `sp + 8`: `r2`
- `sp + 4`: `r1`
- `sp + 0`: `r0` ← `sp` when the handler starts

Crucially, these registers (`r0`–`r3`, `r12`, and `lr`) perfectly match the AAPCS *Caller-Saved* set. Because `r4`–`r11` are *Callee-Saved*, any standard C function invoked during the interrupt will safely preserve them before modifying them. Together, the hardware stacking and the AAPCS software rules guarantee that the interrupted application finds all its registers entirely unmodified upon return. This elegant design allows an interrupt handler to be written as a standard C function (e.g., `void SysTick_Handler(void)`) without requiring specialized assembly wrappers or keywords. Additionally, the hardware ensures the stack pointer remains 8-byte aligned upon entry, adhering to the `STKALIGN` bit configured in the CCR during boot.

1.9.2 Exception Return: `EXC_RETURN`

Inside the exception handler, the `lr` does not contain a standard return memory address. Instead, it holds the `EXC_RETURN` value, which begins with `0xFFFFFF...`—an address region deliberately containing no executable code. When the handler concludes and executes a return instruction (like `bx lr` or `pop {..., pc}`), the CPU hardware intercepts this special value and initiates an **exception return**. It pops the 8-word frame from the stack, reverts to the previous execution mode, and seamlessly resumes at the stacked `PC`.

EXC_RETURN	Return To	Stack Used
0xFFFFFFF9	Thread mode (normal code, e.g., main)	Main stack
0xFFFFFFF1	Handler mode (returning to a preempted handler)	Main stack
0xFFFFFFFD	Thread mode	Process stack

While the Cortex-M supports a secondary Process Stack Pointer, this kernel exclusively uses the Main stack for now. Therefore, returning to Thread mode will consistently utilize `0xFFFFFFF9`. The compiled C handler ends with an ordinary `bx lr`; the magic lies entirely in the value held within the Link Register.

1.9.3 Tail-Chaining and Late Arrival

To optimize back-to-back exceptions, the hardware utilizes two advanced mechanisms that require no software intervention:

- **Tail-chaining:** If a handler completes while another exception is pending, the CPU skips the redundant process of popping the 8-word frame only to immediately push it again. Instead, it branches directly to the pending handler, reducing latency to about 6 cycles (down from 24) on a Cortex-M3.

- **Late arrival:** If a higher-priority exception arrives while the CPU is actively stacking the frame for a lower-priority exception, the CPU simply uses the partially completed frame, executes the higher-priority handler first, and then tail-chains to the lower-priority one.

In both scenarios, the single stacked frame explicitly describes the original code that was preempted.

1.10 The Heartbeat of the OS: The SysTick Timer

To build an operating system capable of measuring time and preempting tasks, a periodic interrupt is mandatory. The **SysTick** timer is a 24-bit down-counter permanently integrated into every Cortex-M core, mapped to the exact same memory addresses across all chips.

1.10.1 Hardware Preemption

If an OS relies entirely on cooperative software, an infinite loop in a user program would permanently hijack the CPU, deadlocking the system. SysTick prevents this through **hardware preemption**. By periodically firing an interrupt (e.g., every 10 ms), the hardware forcefully pauses the current execution and jumps to the kernel's `SysTick_Handler`. This guarantees the OS regains control at fixed intervals to update the system clock, perform scheduling, or terminate rogue tasks. A hardware interrupt is the only mechanism powerful enough to definitively stop the CPU.

1.10.2 The SysTick Hardware Registers

The timer operates by counting down from a configured `LOAD` value to 0, reloading the `LOAD` value on the subsequent clock cycle. Consequently, exactly one period equals LOAD + 1 cycles.

Register	Address	Function
`CTRL`	`0xE000E010`	Bit 0: **ENABLE** the counter. Bit 1: **TICKINT** (raise the SysTick exception at 0). Bit 2: **CLKSOURCE** (1 = processor clock). Bit 16: **COUNTFLAG** (reads 1 if 0 was reached since last read).
`LOAD`	`0xE000E014`	24-bit reload value (maximum `0xFFFFFF`).
`VAL`	`0xE000E018`	Current count. Writing any value resets it to 0.
`CALIB`	`0xE000E01C`	Read-only calibration data. Bits 23–0 (**TENMS**) hold the reload value for 10 ms on the reference clock.

To initiate the timer, three stores are required:

```
SysTick->LOAD = 249999;          /* period: 250,000 cycles (10 ms at 25 M
SysTick->VAL  = 0;               /* start from a clean count */
SysTick->CTRL = SysTick_CTRL_ENABLE | SysTick_CTRL_TICKINT | SysTick_CTRL_
```

The SysTick priority is configured using the `SHPR` byte (set to `0xE0` during boot), ensuring it is the least urgent exception in the system. It requires no interrupt-controller setting and no explicit hardware acknowledgment; its pending state clears automatically when the handler starts. While the `CALIB` register allows portable code to achieve a 10 ms tick without knowing the board speed, we know our processor clock is 25 MHz, so we explicitly calculate the `LOAD` value using `CLKSOURCE = 1`.

1.10.3 Building the SysTick Driver

To integrate this hardware into our OS, we write a device driver, structured similarly to the UART driver. The API exposed in `drivers/systick.h` includes initialization and time-tracking functions:

```
void systick_init(void);
uint32_t systick_uptime_ms(void);
```

Choosing the Tick: Reload Value and Rate

The formula for the reload value is **LOAD = (f / rate) - 1**. With our processor frequency (f) at 25 MHz, if we want an interrupt frequency (rate) of 100 Hz (which equals 100 ticks per second, or one tick every 10 ms), the calculation is:

$$\text{LOAD} = \frac{25,000,000}{100} - 1 = 249,999$$

This value (`0x3D08F`) easily fits within the 24-bit limit of the `LOAD` register.

Choosing the tick rate is a fundamental OS design trade-off:

- A faster tick (e.g., 1 ms / 1,000 Hz) measures time more precisely but forces the CPU to process 1,000 exception entries and exits per second, wasting significant CPU time even when the system is idle.
- A slower tick (e.g., 10 ms / 100 Hz) reduces interrupt overhead but lowers timing resolution.

Our kernel utilizes the classic **10 ms (100 Hz)** rate. In QEMU, this specific rate also has the practical benefit of keeping exact time against a physical wall clock, whereas a 1 ms tick runs approximately 26% slower in real-world time due to emulation overhead.

Uptime in Milliseconds and Wraparound Safety

In `drivers/systick.c`, we maintain a global counter tracking the uptime in milliseconds.

```
#include "drivers/systick.h"
#include "cmsdk.h"

#define CPU_CLOCK_HZ 25000000UL      /* processor clock: 25 MHz */
#define TICK_MS 10

static volatile uint32_t uptime_ms; /* changed by the handler */

void systick_init( void )
{
    uptime_ms = 0;
    SysTick->LOAD = CPU_CLOCK_HZ / ( 1000 / TICK_MS ) - 1;
    SysTick->VAL = 0;                /* any write: counter = 0 */
    SysTick->CTRL = SysTick_CTRL_ENABLE | SysTick_CTRL_TICKINT | SysTick_C
}

void SysTick_Handler( void )
{
    uptime_ms += TICK_MS;            /* no acknowledge needed */
}

uint32_t systick_uptime_ms( void )
{
    return uptime_ms;                /* one 32-bit read: never torn */
}
```

The variable `uptime_ms` is strictly declared `volatile` because the `SysTick_Handler` updates it entirely asynchronously. The getter function `systick_uptime_ms` performs a single 32-bit read, which is architecturally guaranteed to be atomic.

Because `uptime_ms` is a `uint32_t`, it wraps from `0xFFFFFFFF` back to 0 after approximately 49.7 days. To safely measure elapsed time without wraparound bugs, the system must *always* use unsigned subtraction:

```
if (now - start >= delay) { /* Safe */ }
```

If `start = 0xFFFFFF00` and the counter wraps such that `now = 0x00000064` (100), the calculation `now - start` safely evaluates to `0x164` (356 ms) thanks to modulo 2^{32} arithmetic.

`wfi`: An Idle Loop That Sleeps

With the hardware timer firing every 10 ms, we can efficiently halt the CPU when it has no immediate work using the `wfi` (Wait For Interrupt) assembly instruction.

```
for ( ;; ) {
    __asm volatile ( "wfi" );  /* sleep until the next interrupt */
    /* ... check whether something has to be done ... */
}
```

The `wfi` instruction instantly puts the processor into a low-power sleep state until any exception is pending. Instead of a polling loop running millions of useless checks per second, a `wfi` loop executes exactly once per interrupt, ensuring the OS remains highly responsive while saving power.

1.11 The Nested Vectored Interrupt Controller (NVIC)

While SysTick is an internal core exception, external peripheral interrupts (like the UART or timers on the APB bus) are managed by a memory-mapped I/O device called the **NVIC** (Nested Vectored Interrupt Controller).

The NVIC sits between the peripherals and the CPU core as a gatekeeper and priority manager. On this board, it handles 32 lines (**IRQ 0 to IRQ 31**). For example, UART0 RX is IRQ 0 and UART0 TX is IRQ 1.

Recalling Unit 1 (**IRQ n maps to exception** $16 + n$), the handler address resides at $4 \times (16 + n)$. For UART0 TX (IRQ 1), this is slot 17, at address `0x44`.

The NVIC is controlled via specific memory-mapped registers:

Register	Address	Function (For IRQ n)
`ISER`	`0xE000E100`	**Interrupt Set-Enable.** Writing a 1 to bit n (mod 32) of word $n/32$ enables the IRQ line. Writing 0 has no effect.
`ICER`	`0xE000E180`	**Interrupt Clear-Enable.** Writing a 1 disables the IRQ line.
`ISPR`	`0xE000E200`	**Interrupt Set-Pending.** Writing a 1 manually marks the interrupt as pending (simulating a hardware request).
`IPR`	`0xE000E400`	**Interrupt Priority.** Configures priority level. Only the upper bits count; a lower number means a higher urgency.

To configure priority and enable UART0 TX, we execute two simple stores. Because writing a 0 to `ISER` changes nothing, we do not need a read-modify-write pattern:

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;
/* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable I
```

1.11.1 The Three Gates and the Golden Rule

An external device's interrupt will only fire if three conditions—the *three gates*—are simultaneously true:

1. **The Device Gate:** The peripheral is configured to generate interrupts (e.g., `TXIRQEN` or `RXIRQEN` in the UART's `CTRL` register).
2. **The NVIC Gate:** The specific IRQ line is unmasked (via `ISER`).
3. **The CPU Gate:** The global mask is off (`PRIMASK` = 0) and the interrupt priority is strictly more urgent than the currently running code.

Missing either of the first two gates produces the same symptom: nothing happens.

The Golden Rule: Acknowledge at the Device

NVIC lines are **level-sensitive**: a device keeps its line active as long as its internal interrupt flag is set (e.g., bit 0 of UART `INTSTATUS`). This introduces the Golden Rule: **software must acknowledge and clear the interrupt at the device level before returning**.

If the handler executes `bx lr` without clearing the hardware flag, the NVIC instantly fires the interrupt again, permanently trapping the OS in an infinite loop.

Acknowledging usually means reading `DATA` or writing a 1 to a clear bit. `INTSTATUS` uses write-1-to-clear logic: `CMSDK_UART0->INTSTATUS = UART_INT_TX;` clears bit 0 and leaves the rest alone. (SysTick, being a core exception, requires no explicit hardware acknowledgment).

Shared Interrupt Lines (Q&A)

While hardware designers typically map one device to one line, complex boards might wire multiple devices to the same IRQ. Because the CPU only sees a single asserted wire, the software handler must manually poll the status registers of *every* device sharing that line to find the trigger, service it, and clear the flag.

1.12 Expanding the Vector Table for External Devices

The base's `boot/startup.s` allocates 16 core slots plus 32 hardware IRQs, totaling 48 words (192 bytes, or `0xC0`).

Devices are assigned specific names (e.g., `UART0TX_Handler` is IRQ 1 at word 17; `TIMER0_Handler` is IRQ 8 at word 24). Unused slots point to a `Default_Handler`. Each handler name uses `.weak` and `.thumb_set` directives: this allows a C function defined with the identical name to automatically override the weak default handler.

You can verify this in GDB: examining `x/4wx 0x40` (slots 16–19) reveals identical words pointing to `Default_Handler` (plus 1 for the Thumb state).

1.13 Concurrency and Shared Data

1.13.1 Data Shared by a Handler and `main`

An interrupt handler and `main` frequently share data: the handler produces it and `main` consumes it, or vice versa. While the `volatile` keyword forces the compiler to physically read and write memory on every access, it does **not** make an update operation (like `count++`) indivisible.

At the assembly level, an increment involves three distinct steps:

```
ldr  r1, [r0]      /* r1 = count              (say 5) */
adds r1, r1, #1    /* r1 = 6   <- the handler runs here, count = 6 */
str  r1, [r0]      /* count = 6: the handler's update is lost */
```

An interrupt can arrive between any two instructions. If the handler modifies `count` between `main`'s load and `main`'s store, `main` subsequently overwrites it with a stale value. This scenario is a **lost update**, a classic example of a **race condition** where the outcome depends entirely on exact timing.

What is safe without help: a single, aligned 32-bit read or write of a value (e.g., reading `uptime_ms`). What is *not* safe: any read-modify-write operation (`++`, `+=`, `|=`), and updates involving several related variables that must remain consistent.

1.13.2 Critical Sections with `irq_save` and `irq_restore`

A **critical section** is a short segment of code that no handler is permitted to interrupt. On a single-core CPU, temporarily masking interrupts is sufficient:

```
uint32_t s = irq_save();      /* PRIMASK = 1: no handler can run from here
count++;                      /* ... so load, add and store happen together
irq_restore( s );             /* ... to here; back to the previous mask */
```

Three critical rules accompany this pattern:

1. **Asymmetry:** Only the side that can be interrupted requires it. `main` can be interrupted by the handler, but not vice versa, so the protection belongs in `main`.

2. **Brevity:** Keep the critical section extremely short and never wait inside it. Any interrupt arriving in the meantime is safely delayed (kept pending by the hardware) until `irq_restore`.

3. **Proper Nesting:** Always use `irq_restore(s)` rather than a raw `cpsie i` instruction. This ensures proper nesting if the caller had already masked interrupts prior to entering the block.

1.13.3 A Ring Buffer

To send a long message via interrupts, `main` must leave the bytes in memory for the handler to fetch asynchronously. The classic structure for this is a **ring buffer**: an array utilized as a circular queue.

The producer stores data at the `head` index, the consumer extracts data at the `tail` index, and both indices wrap around to 0 upon reaching the end of the array.

```
next = ( i + 1 ) & ( SIZE - 1 );                    /* = (i + 1) % SIZE when S
empty:  head == tail
full:   ( ( head + 1 ) & ( SIZE - 1 ) ) == tail /* one slot always stays
```

Restricting `SIZE` to a power of two (e.g., 64) optimizes the modulo operation (`%`) into a cheap bitwise AND (`&`) requiring no division. Furthermore, this lock-free design ensures each index has exactly one writer: `head` is written exclusively by the producer, and `tail` exclusively by the consumer.

With `SIZE = 64`, the maximum number of bytes that can wait is **63**. If all 64 slots were used, a full buffer would result in `head == tail`, making "full" structurally indistinguishable from "empty". By deliberately leaving one slot permanently unused, the two states remain mathematically distinct.

1.13.4 Interrupt-Driven Transmission

With the ring buffer in place, we expand our UART driver capabilities to handle multiple characters efficiently via interrupts, rather than sending single characters through a polling loop.

The design has two sides: a producer and a consumer.

The Producer: `uart_write`

The producer function, `uart_write(buf, len)`, is called by the main application. It receives a buffer containing a string of characters.

Inside a critical section (protected by `irq_save` and `irq_restore`), it checks the available space in the internal ring buffer, copies as many bytes as possible, and returns the number of bytes successfully accepted. If the buffer fills up, it stops copying and simply returns the count; the caller is responsible for retrying with the rest.

Crucially, it implements the **Priming Rule**: if the transmitter is currently idle (meaning `tx_busy` is 0), the producer must send the first byte manually to kickstart the hardware, because no interrupt is coming to do it. The rule dictates that we must *first* enable the TX interrupt (`TXIRQEN`) and *then* write the first byte to the `DATA` register. Turning the interrupt on *while* the transmitter is idle raises nothing, and turning it on *after* the write misses the hardware event, leaving the transmission permanently stalled.

```
unsigned uart_write( const char *buf, unsigned len )
{
    unsigned accepted = 0;
    uint32_t saved = irq_save(); /* the handler must not run during our u

    while ( accepted < len && !tx_full() )
    {
        tx_buf[tx_head] = buf[accepted];
        tx_head = ( tx_head + 1 ) & ( TX_BUF_SIZE - 1 );
        accepted++;
    }

    if ( !tx_busy && !tx_empty() ) /* idle: no IRQ will come, so prime it
```

```
    {
        CMSDK_UART0->CTRL |= UART_CTRL_TXIRQEN; /* FIRST enable TX interr
        while ( CMSDK_UART0->STATE & UART_STATE_TXBF ) {} /* wait if poll

        CMSDK_UART0->DATA = (uint32_t)(unsigned char) tx_pop(); /* THEN
        tx_busy = 1;
    }

    irq_restore( saved );
    return accepted;
}
```

The Consumer: `UART0TX_Handler`

The consumer is the `UART0TX_Handler`. It runs automatically every time a byte successfully leaves the hardware.

Because it is an interrupt handler, it needs no critical section (while it runs, `main` cannot run). It first acknowledges the interrupt at the device by writing to the `INTSTATUS` register (using a Write-1-to-Clear operation). It then checks if the buffer is empty. If it has data, it sends the next byte from the tail of the buffer. If the buffer is empty, it turns the `TXIRQEN` interrupt off and marks the transmitter as idle (`tx_busy = 0`).

```
void UART0TX_Handler( void )
{
    CMSDK_UART0->INTSTATUS = UART_INT_TX; /* acknowledge at the device (W

    if ( !tx_empty() ) {
        CMSDK_UART0->DATA = (uint32_t)(unsigned char) tx_pop(); /* send n
    } else {
        CMSDK_UART0->CTRL &= ~UART_CTRL_TXIRQEN;   /* nothing left: interr
        tx_busy = 0;                               /* transmitter idle */
    }
}
```

Initialization and Priority

During `uart_init`, we establish the buffer variables and configure the NVIC priority. We assign the UART TX IRQ a priority of `0xC0`. Since SVCall operates at `0x80` and SysTick at `0xE0`, this ensures device interrupts are less urgent than system calls but more urgent than the periodic timer.

```
    /* Start with an empty buffer and an idle transmitter. */
    tx_head = 0; tx_tail = 0; tx_busy = 0;

    /* Priority 0xC0: less urgent than SVCall (0x80), more than SysTick (
    NVIC->IPR[UART0TX_IRQn] = 0xC0;
    /* Enable IRQ 1 in the NVIC (write-1-to-enable). */
    NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 );
```

The device switch (`TXIRQEN`) stays deliberately off during initialization; `uart_write` will turn it on only when there is actually data to send.

Testing the Ring Buffer: Sending 160 Bytes

To verify the interrupt-driven transmission, we can send a message significantly longer than the 64-byte buffer.

In `src/main.c`, we define a 160-byte test string:

```
/* 4 lines x 40 bytes = 160 bytes: more than the 64-byte buffer */
static const char msg[] =
    "line 1: the quick brown fox jumps over.\n"
    "line 2: the quick brown fox jumps over.\n"
    "line 3: the quick brown fox jumps over.\n"
    "line 4: the quick brown fox jumps over.\n";
```

We then implement a loop to push this data to the kernel:

```
const char *p = msg;
unsigned left = sizeof msg - 1;          /* 160: without the final '\0' */
while ( left > 0 )
{
    unsigned n = uart_write( p, left ); /* copy what fits */
    p += n;
    left -= n;
    if ( n == 0 )
    {
        __asm volatile ( "wfi" );        /* buffer full: sleep until an IR
    }
}
```

Execution Analysis When executed, `uart_write` is called exactly 3 times, returning **63, 63, and 34** (which perfectly sums to 160). Each call fills the 63 usable slots of the ring buffer. Because QEMU's UART emulator sends bytes instantly, the handler empties the buffer the moment `irq_restore` unmasks interrupts, meaning `n == 0` never actually evaluates to true in this specific emulation. On real physical hardware at 115,200 baud, the loop would frequently sleep via `wfi`.

The consumer, `UART0TX_Handler`, runs exactly **160 times** (once per byte sent, regardless of whether it was primed or completely interrupt-driven). On the final run, finding the buffer empty, it automatically disables `TXIRQEN`.

Verification via GDB You can explicitly verify this internal behavior using GDB:

- **Return values:** Set `break uart_write`, execute `continue`, and use `finish` three times to inspect the return values (`$1 = 63`, `$2 = 63`, and `$3 = 34`).
- **Handler hits:** Set `break UART0TX_Handler` and use `ignore 1 100000` to let it run in the background without stopping. After setting a temporary breakpoint (`tbreak`) on a subsequent line in `main` and continuing, `info breakpoints` will explicitly confirm the handler was hit exactly 160 times.

- **NVIC state:** Examining `x/2wx 0xE000E100` shows `0x00000002` (confirming IRQ 1 is enabled in `ISER`), and `x/4bx 0xE000E400` confirms `IPR` byte 1 holds `0xC0`.

End of Unit 2

At this stage, `drivers/uart.c`, `drivers/systick.c`, `kernel/irq.s`, and `boot/startup.s` are functionally complete.

The `src/main.c` file currently initializes the UART, prints the kernel banner, starts the SysTick timer, sends the 160-byte test message through the interrupt-driven ring buffer, and then runs the temporary privileged loop that prints the uptime every second.

Currently, everything still runs privileged, directly inside `main`. The final architectural phase (Unit 3) will move this application workload to an unprivileged environment and establish a real, secure kernel boundary between the user application and the hardware.

1.14 Unit 3: System Calls and the Kernel Boundary

1.14.1 Recap: Privilege and `svc`

From Week 2, we established four architectural facts:

1. `main` starts **privileged**. Setting `CONTROL.nPRIV = 1` drops the CPU into **unprivileged** Thread mode. This transition is one-way; unprivileged code cannot clear the bit again.

2. Unprivileged code attempting to touch the System Control Space (SysTick, NVIC, SCB) triggers a **BusFault**.

3. The `svc #n` instruction is always permitted and raises the **SVCall** exception.

4. Every exception handler runs in Handler mode, which is **privileged**. When the handler returns, the CPU resumes execution right after the `svc` instruction, dropping back to unprivileged mode.

In Week 2, `SVC_Handler` performed one fixed job (printing a single message). In this unit, it becomes a **system-call interface** capable of multiplexing several services, receiving arguments, and returning results to user space.

1.14.2 The Rule From Now On: The App Asks, The Kernel Does

In our architecture, the application never calls a device driver directly and never touches a hardware register. **Every I/O operation crosses the kernel boundary via a system call**, just as in Linux a user-space `printf` ultimately invokes the `write` system call.

The end-to-end structure is:

```
 app_main()          printf("uptime: %u ms\n", now)        src/app.c          \
     |                                                                          | un
 user library        printf -> write_all -> sys_write()    lib/printf.c       | (Th
     |                                                                          |
 user stub           svc #1 ; bx lr                        lib/syscalls.s     /
=====+============== SVCall exception: the kernel boundary ======================
 entry               SVC_Handler: mrs r0, msp ; b ...      kernel/svc_entry.s  \
     |                                                                          | p
 dispatcher          svc_dispatch(frame): table[n](...)    kernel/syscall.c    | (H
     |                                                                          |
 driver              uart_write() + UART0TX_Handler        drivers/uart.c      /
     |
 hardware            UART0 registers at 0x40004000
```

Logical vs. Physical Protection

Because our Cortex-M3 setup does not yet use a Memory Protection Unit (MPU), the hardware blocks unprivileged access only to the System Control Space; the UART registers at `0x40004000` remain reachable from unprivileged code.

However, the kernel boundary is a hard technical requirement: the UART driver uses `irq_save()` (which executes `cpsid i`) to protect its ring buffer. Because `cpsid` is silently ignored in unprivileged mode, the driver is unsafe unless executed in privileged kernel mode.

1.14.3 The System-Call ABI and Services

An Application Binary Interface (ABI) is an agreement at the level of registers and machine instructions rather than C source code:

- **Service Number:** Embedded inside the `svc #n` instruction.
- **Arguments:** Passed in registers `r0`, `r1`, and `r2`, following the AAPCS.
- **Result:** Returned to the application in `r0`, as expected by a C caller.

To avoid magic numbers in C code, symbolic constants (`SYS_WRITE`, `SYS_READ`, `SYS_UPTIME`) and prototypes are defined in a shared header (`lib/syscalls.h`) included by both user space and the kernel:

Number	Service	Arguments	Result
1	`sys_write`	`buf`, `len`	Bytes **accepted** (0 to `len`). Routes to `uart_write`.
2	`sys_read`	None	One received byte, or `-1` if none. Routes to `uart_getc_polled`.
3	`sys_uptime`	None	Milliseconds since boot. Routes to `systick_uptime_ms`.
0, ≥ 4	Invalid	None	Unknown service (e.g., 0 or 99): returns `-1`.

No service waits: `sys_write` takes only what currently fits in the TX ring buffer and returns the count immediately. The unprivileged caller is responsible for waiting.

1.14.4 User Space Implementation: The Stubs

Each system call is exposed to C via a two-instruction assembly **stub** in `lib/syscalls.s`. By the AAPCS, the C caller has already placed arguments into `r0` and `r1`. The `svc` instruction traps into the kernel, where hardware stacks them as `frame[0]` and `frame[1]`. Upon exception return, `r0` holds the result written by the kernel into `frame[0]`, and `bx lr` returns it unchanged.

Because the number after `svc` is encoded directly into the instruction opcode, it must be a literal (`1`, `2`, or `3`). The assembly file does not read the C macros from `lib/syscalls.h`, so these numbers must be kept in sync manually.

```
    .syntax unified
    .cpu cortex-m3
    .thumb
    .section .text,"ax",%progbits

/* int sys_write(const char *buf, unsigned len): buf in r0, len in r1 */
    .global sys_write
    .type sys_write, %function
    .thumb_func
sys_write:
    svc #1                          /* svc(SYS_WRITE, buf, len) */
    bx lr                           /* return r0 (set by the kernel in frame[
    .size sys_write, . - sys_write

/* int sys_read(void) */
    .global sys_read
    .type sys_read, %function
    .thumb_func
sys_read:
    svc #2                          /* svc(SYS_READ) */
    bx lr                           /* return r0 */
    .size sys_read, . - sys_read

/* uint32_t sys_uptime(void) */
    .global sys_uptime
    .type sys_uptime, %function
    .thumb_func
sys_uptime:
    svc #3                          /* svc(SYS_UPTIME) */
    bx lr                           /* return r0 */
    .size sys_uptime, . - sys_uptime
```

1.14.5 The Kernel Entry Point: Why It Must Be Assembly

When `svc` executes, it traps into `SVC_Handler`, which passes a pointer to the 8-word hardware exception frame (`uint32_t *frame`) to the C dispatcher (`svc_dispatch`).

`SVC_Handler` must be written in assembly (`kernel/svc_entry.s`). If written in C, the compiler may insert prologue instructions (such as `push {r7, lr}`) before the first C line runs, and this behavior changes depending on optimization flags like `-O0` versus `-O2`. Modifying `sp` in the prologue shifts the stack pointer and corrupts the offset to the exception frame.

```
    .global SVC_Handler
    .type SVC_Handler, %function
    .thumb_func
SVC_Handler:
    mrs r0, msp                 /* r0 = MSP: frame = address of the 8 sav
    b svc_dispatch              /* svc_dispatch(frame); lr still = EXC_RE
```

Executing `mrs r0, msp` first captures the exact Stack Pointer left by hardware and passes it as argument `r0` to `svc_dispatch`. We use a plain branch (`b`) rather than branch-with-link (`bl`): `bl` would overwrite `lr` and destroy the `EXC_RETURN` value (`0xFFFFFFF9`). With a tail branch (`b`), `lr` is preserved so that returning from `svc_dispatch` triggers the exception return directly.

1.14.6 Extracting the Service Number and Arguments

In Thumb mode, `svc #n` is a **16-bit** instruction: the high byte is the opcode `0xDF` and the low byte is the immediate `n` (so `svc #1` is `0xDF01`). Because the Cortex-M3 is **little-endian**, the low byte `n` is stored at the lower address, followed by `0xDF`:

```
address:    0x8e0    0x8e1    0x8e2    0x8e3
byte:       0x01     0xdf     0x70     0x47
            +- svc #1 --+     +- bx lr ---+
```

On exception entry, hardware pushes the 8-word frame (`r0-r3`, `r12`, `lr`, `pc`, `xPSR`). The stacked `PC` (`frame[6]`) points to the instruction immediately *after* `svc` (`bx lr`). Thus, the opcode `0xDF` sits at `PC - 1` and the service number `n` sits at `PC - 2`. The user arguments (`r0-r2`) are read from `frame[0]`, `frame[1]`, and `frame[2]`.

1.14.7 The Dispatcher and Validating the Result Return

To route calls without a large `switch`, `kernel/syscall.c` uses a function pointer table:

```
typedef uint32_t ( *syscall_fn )( uint32_t a0, uint32_t a1, uint32_t a2 )

static const syscall_fn table[] = {
    [SYS_WRITE]   = sys_write_impl,
    [SYS_READ]    = sys_read_impl,
    [SYS_UPTIME] = sys_uptime_impl,
};
#define TABLE_SIZE ( sizeof( table ) / sizeof( table[0] ) )
```

```
void svc_dispatch( uint32_t *frame )
{
    uint8_t n = ( ( uint8_t * ) frame[6] )[-2];

    if ( n >= TABLE_SIZE || table[n] == 0 ) {
        frame[0] = ( uint32_t ) -1;              /* unknown service */
        return;
    }
    /* run it; the result goes into the stacked r0 = the caller's r0 */
    frame[0] = table[n]( frame[0], frame[1], frame[2] );
}
```

Why Write to `frame[0]`? The kernel cannot return the result simply by leaving it in physical register `r0`. On exception return, the hardware pops the 8-word frame from the stack back into the CPU registers, overwriting physical `r0` with `frame[0]`. Writing the return value into `frame[0]` ensures the hardware restores it into the caller's `r0`.

Try It: Your First System Call

In `src/main.c`, include `"lib/syscalls.h"` and replace `systick_uptime_ms()` with `sys_uptime()`. The output is unchanged, but follows the new path: `sys_uptime` → `svc #3` → `SVC_Handler` → `svc_dispatch` → `sys_uptime_impl` → `systick_uptime_ms`. (Although `main` is still privileged here, `svc` works from privileged mode too.)

Finding the Service Number in GDB

You can inspect the hardware frame and decode n in GDB:

```
(gdb) break *SVC_Handler                  # very first instruction of the e
(gdb) continue
(gdb) x/8wx $sp                           # the frame: r0 r1 r2 r3 / r12 lr
(gdb) p/x $lr
(gdb) x/2bx ((unsigned *)$sp)[6] - 2      # the 2 bytes before the stacked
```

Here `frame[6]` points right after `svc` (e.g., `0x7e6`), and the two preceding bytes are `0x03` and `0xDF` (`svc #3` in little-endian, so $n = 3$, `SYS_UPTIME`). Moreover, `lr` holds `0xFFFFFFF9` (`EXC_RETURN`), preserved by `b svc_dispatch`. Running `stepi` twice enters `svc_dispatch`, and `finish` returns directly to `bx lr` in `sys_uptime`.

1.14.8 User Space Printing and Hardware Wait States

While the kernel uses polled `kprintf`, user applications use `printf` built on system calls. Because `sys_write` pushes data into the ring buffer and immediately returns the count accepted, user-space `printf` loops until all bytes are sent. When the buffer is full, the application executes `wfi` (Wait For Interrupt), putting the CPU to sleep until a hardware interrupt (such as UART TX freeing buffer space) wakes it up.

1.14.9 The Kernel Rules: Where SVC Works

The `svc` instruction raises `SVCall` synchronously, but an exception can only preempt code that is *less* urgent than itself:

Where `svc` runs	**Current priority**	**Result**
`app_main`, `main` (Thread mode)	none (lowest)	taken
`SysTick_Handler` / `UART0TX_Handler`	`0xE0` / `0xC0`	taken: SVCall is more urgent, so it nests
`SVC_Handler` and anything it calls	`0x80`	**HardFault**
a fault handler	`0x00` or more urgent	**HardFault**
between `irq_save` and `irq_restore` (`PRIMASK = 1`)	masked	**HardFault**

Because `SVCall` has priority `0x80`, executing `svc` from a handler of equal or higher urgency (or while `PRIMASK = 1`) cannot be serviced and escalates to a **HardFault**. This is why the kernel retains its own polled `kprintf` independent of system calls.

1.14.10 Why the Kernel Never Waits

The kernel must never wait inside an exception handler. If `sys_write_impl` looped inside `SVCall` (priority `0x80`) waiting for the UART TX buffer to empty, the less urgent UART TX interrupt (priority `0xC0`) could never preempt it, deadlocking the system. SysTick (`0xE0`) would also be blocked, stopping the system clock. Instead, the kernel accepts only what fits in the buffer and returns immediately, leaving waiting (`wfi`) to unprivileged Thread mode.

1.14.11 User Space Library: Implementing `printf`

With the system calls operational, we shift our focus to user space and implement `printf` as a user-level library in `lib/printf.c`.

Unlike the kernel's `kprintf` (which calls the polled hardware driver directly), the user-level `printf` operates under unprivileged constraints in two steps:

1. It uses the shared `vformat` engine to format the string into a local 64-byte buffer allocated on the unprivileged task's stack.

2. It flushes this buffer to the hardware by invoking the `sys_write` system call.

Handling Partial Writes and Waiting (`write_all`)

Because `sys_write` pushes data into the kernel's 64-byte interrupt-driven ring buffer, it only accepts what currently fits and returns the accepted count immediately without waiting.

The user library handles partial writes via `write_all`, which loops until all bytes are sent. If `sys_write` returns 0, the ring buffer is full: instead of busy-waiting, the

application executes `wfi` (Unit 2). The CPU sleeps until an interrupt fires (such as UART TX freeing a buffer slot), then resumes the loop:

```
void write_all(const char *buf, unsigned len) {
    while ( len > 0 ) {
        int n = sys_write(buf, len); /* The kernel takes what fits */
        buf += n;                    /* Skip the accepted bytes */
        len -= n;

        if (n == 0) {
            __asm volatile ("wfi");  /* Buffer full: sleep until an inter
        }
    }
}
```

1.14.12 Implementing `sleep_ms` in User Space

Using `wfi` and unsigned subtraction (Unit 2), we can implement `sleep_ms` without imprecise delay loops by polling `sys_uptime()` (`SYS_UPTIME`) between sleep states:

```
void sleep_ms(uint32_t ms) {
    uint32_t start = sys_uptime();
    while (sys_uptime() - start < ms) {
        __asm volatile ("wfi"); /* Sleep until the next interrupt */
    }
}
```

The condition `sys_uptime() - start < ms` uses unsigned 32-bit arithmetic, which handles 32-bit wraparound (after $\approx$ 49.7 days) without error. Because the waiting occurs in unprivileged Thread mode, the kernel is never blocked: every interrupt can preempt the sleeping task, allowing the OS to handle background UART transmission and timekeeping.

1.14.13 The Final System Boot Flow

`src/main.c` acts as the **kernel** initializer. It runs privileged once at boot and performs five steps:

1. `uart_init()`: Initializes the console first so subsequent code can print.
2. `kprintf`: Prints the boot banner via polling.
3. `systick_init()`: Starts the 10 ms tick used by `sys_uptime` and `sleep_ms`.
4. **Privilege Drop**: Sets `CONTROL.nPRIV = 1` followed by an `isb` (Instruction Synchronization Barrier). The `isb` flushes any instructions already prefetched under the old privileged state, ensuring all subsequent instructions run unprivileged.
5. `app_main()`: Calls the user application, which never returns.

```
#include "drivers/uart.h"
#include "drivers/systick.h"
#include "kernel/kprintf.h"
#include "src/app.h"

int main( void )
{
    uart_init();                                    /* 1. console first */
    kprintf( "scratch-os: kernel started\n" );      /* 2. polled print */
    systick_init();                                 /* 3. 10 ms tick */
    kprintf( "dropping to unprivileged mode\n" );

    __asm volatile (                /* 4. CONTROL = CONTROL | 1; (nPRIV = 1) */
        "mrs r0, control    \n"     /* r0 = CONTROL */
        "orr r0, r0, #1     \n"     /* r0 |= 1: set nPRIV */
        "msr control, r0    \n"     /* CONTROL = r0: now unprivileged */
        "isb                \n"     /* refetch under the new rules */
        : : : "r0", "memory" );

    app_main();                                     /* 5. never returns */
    for ( ;; ) { }                                  /* not reached */
}
```

1.14.14 The User Application: `app_main`

`src/app.c` contains the unprivileged application, which includes only `lib/syscalls.h` and `lib/printf.h`.

Every 1000 ms, it prints the uptime. It compares against the system clock (`now - last >= 1000`) rather than counting loop iterations, which would drift due to the time spent inside `printf`. On each iteration, it also checks `sys_read()` for incoming keys, echoes them if present, and sleeps via `sleep_ms(10)`:

```
#include <stdint.h>
#include "src/app.h"
#include "lib/syscalls.h"
#include "lib/printf.h"

void app_main( void )
{
    printf( "app: hello from unprivileged code\n" );
    uint32_t last = 0;          /* when the uptime was last printed */

    for ( ;; )
    {
        uint32_t now = sys_uptime();
        if ( now - last >= 1000 )           /* one second passed? */
        {
            printf( "uptime: %u ms\n", now );
            last += 1000;
```

```
        }

        int c = sys_read();                    /* never waits: -1 = no key *
        if ( c != -1 )
        {
            printf( "key: %c\n", c );
        }

        sleep_ms( 10 );
    }
}
```

While `app_main` runs unprivileged, hardware interrupts (SysTick, UART TX) and system calls (SVCall) still execute privileged: the Cortex-M hardware raises privilege on exception entry and restores unprivileged mode on return.

1.14.15 A Debug Print Inside the Kernel

This experiment answers a question: why does the kernel keep its own `kprintf`, now that there is a `printf`? Suppose you want to trace every uptime request. In `kernel/syscall.c`, add `#include "lib/printf.h"` below the other includes, and in `sys_uptime_impl` add one line before the `return`:

```
    printf("uptime called\n");          /* NEW: debug print */
```

You might expect `uptime called` about a hundred times per second. `make run` prints instead:

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
HardFault: HFSR=0x40000000
```

Follow the call: `printf` → `write_all` → `sys_write` → `svc #1`, executed **inside** the SVCall handler, whose priority is `0x80`. An exception can only interrupt code that is **less urgent** than itself, and SVCall is not more urgent than SVCall. Unlike an interrupt, an `svc` cannot be postponed until later: it must be taken now or never. So the CPU **escalates** it to a HardFault.

`HFSR` bit 30, **FORCED** (`0x40000000`, `SCB_HFSR_FORCED` in `include/cmsdk.h`), means "a fault was escalated to HardFault". Notice that the report itself prints fine: `HardFault_Handler` uses `kprintf`, which needs no `svc`. Undo both edits (the `#include` and the `printf` line).

1.14.16 Making the Kernel Wait

The second experiment answers another question: why does `sys_write` return a count instead of simply waiting until everything is sent? Make two temporary edits. In `kernel/syscall.c`, in `sys_write_impl`, replace `return uart_write(buf, len);` with a loop that waits until **everything** is accepted:

```
    unsigned done = 0;
    while (done < len)
```

```
        done += uart_write(buf + done, len - done);
    return done;
```

And in `src/app.c`, in `app_main`, right after the hello `printf`, add a direct write of 130 bytes, more than the 64-byte ring buffer (the compiler joins the four adjacent strings into one):

```
    sys_write("0123456789012345678901234567890123456789"
              "0123456789012345678901234567890123456789"
              "0123456789012345678901234567890123456789"
              "012345678\n", 130);
```

Do the 130 characters appear? Do the `uptime:` lines? `make run` prints:

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
0
```

One `0`, then nothing, forever. In GDB (`make debug`, `continue`, then `Ctrl-C` to interrupt), `p $xpsr & 0x1ff` gives `11`: the CPU is inside the SVCall handler. `bt` shows `uart_write` ← `sys_write_impl` ← `svc_dispatch` ← `<signal handler called>` ← `sys_write` ← `app_main` (the innermost frame may be `irq_save` or `irq_restore` instead, depending on where you interrupt it).

Here is what happened. The first `uart_write` filled the ring buffer and **primed** the UART with the first byte: that is the `0` you see. From then on the buffer stays full. Only `UART0TX_Handler`, at priority `0xC0`, can empty it, but the CPU is inside the SVCall handler, at `0x80`, which is more urgent. The TX interrupt stays **pending** forever, and the loop waits forever for room that never comes. SysTick, at `0xE0`, is blocked too: the clock stops.

The fix is the design you already have. The kernel **never waits**: `sys_write` returns the accepted count, and `write_all` does the waiting with `wfi` in **Thread mode**, where the TX interrupt can run. Undo both edits.

1.14.17 Privilege and Layers in GDB

We can verify the privilege separation in GDB by setting breakpoints in `main` and `app_main` and inspecting `$control`:

```
(gdb) break main
(gdb) break app_main
(gdb) continue          -> Breakpoint 1, main ()
(gdb) p/x $control      -> 0x0          (privileged)
(gdb) continue          -> Breakpoint 2, app_main ()
(gdb) p/x $control      -> 0x1          (nPRIV set: unprivileged)
```

Next, place a breakpoint inside `sys_write_impl`:

```
(gdb) break sys_write_impl
(gdb) continue
(gdb) p/x $control
(gdb) p/x $xpsr & 0x1ff
(gdb) bt
```

Inside `sys_write_impl`, `$control` is still `0x1` (`nPRIV` remains set); the CPU has privileged access because it is running in Handler mode. The low 9 bits of `xPSR` hold the exception number, where `0xb` (11) identifies SVCall.

The backtrace (`bt`) shows every layer of the call path:

```
#0  sys_write_impl (a0=..., a1=34, a2=...)
#1  svc_dispatch (frame=...)
#2  <signal handler called>
#3  sys_write ()
#4  write_all (buf=... "app: hello from unprivileged code\n", len=34)
#5  printf (fmt=...)
#6  app_main ()
#7  main ()
```

The `<signal handler called>` frame is GDB's marker for an **exception entry**: frames above it run in privileged Handler mode, while frames below it belong to the unprivileged user application.

1.15 Where to Find the Code

`week-03/` holds two sibling folders, `base/` and `solution/`, each with its own `Makefile` and building independently. All paths in these notes are relative to `week-03/solution/`. The base's `README.md` lists, folder by folder, what is given and what you write across the three units.

The starting point is `week-03/base/`: the full `Makefile` (`run`, `debug`, `run-trace`), the linker script, the complete `include/cmsdk.h`, `boot/startup.s` with CPU setup and the 16 core slots, silent fault handlers in `kernel/faults.c`, every API header, the number helpers in `lib/format.c`, a `src/main.c` that prints nothing, and marker files for everything else.

The finished result is in `week-03/solution/`, organized by OS layer:

- `boot/startup.s`: the 48-entry vector table `isr_vector`, the weak defaults, and `Reset_Handler`;
- `include/cmsdk.h`: the hardware register map (Unit 1);
- `drivers/uart.c` (**driver**): `uart_init`, `uart_putc_polled`, `uart_getc_polled`, the TX ring buffer, `uart_write`, and `UART0TX_Handler`;
- `drivers/systick.c` (**driver**): `systick_init`, `SysTick_Handler`, and `systick_uptime_ms`;
- `lib/format.c`: `vformat`, shared by both print functions;
- `kernel/kprintf.c`: `kprintf`, the kernel's polled print;
- `kernel/faults.c`: the four fault handlers that print `HFSR` or `CFSR`;
- `kernel/irq.s`: `irq_save` and `irq_restore`;
- `kernel/svc_entry.s` (**kernel entry**): the two-instruction `SVC_Handler`;
- `kernel/syscall.c` (**dispatcher and services**): the syscall implementations, `table`, and `svc_dispatch`;

- `lib/syscalls.s` and `lib/syscalls.h` (**user stubs**): `sys_write`, `sys_read`, and `sys_uptime` (`svc #n ; bx lr`) plus service numbers;
- `lib/printf.c` (**user library**): `write_all`, `printf`, and `sleep_ms`;
- `src/main.c` (**kernel boot**): initializes hardware, drops privilege, and calls `app_main`;
- `src/app.c` (**application**): `app_main`, the unprivileged user task.

Every file in `solution/` is commented line by line. If these notes and the code ever disagree, the code is the reference.